



UNIVERSIDADE
DE SANTIAGO



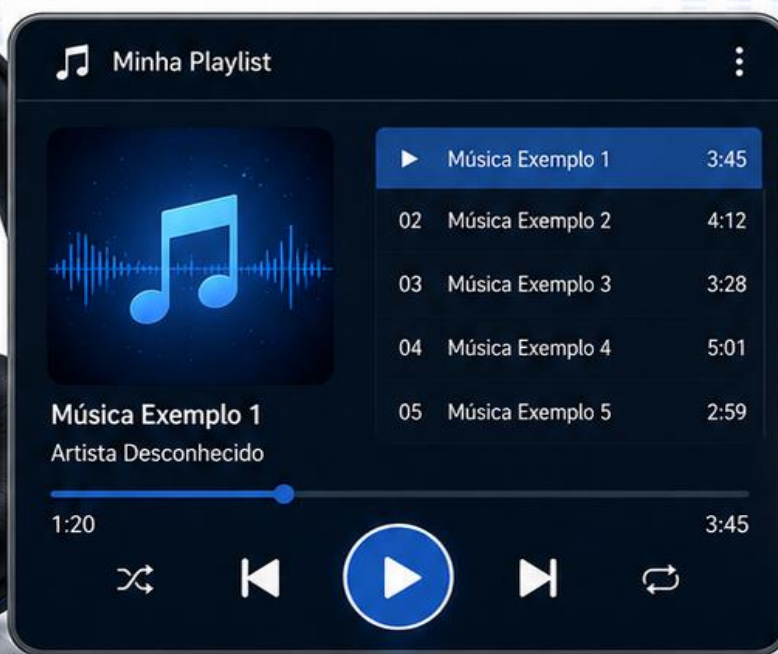
CURSO DE INFORMÁTICA

RELATÓRIO

TRABALHO DE TURMA

LEITOR DE MP3

Leitor de MP3 em Java utilizando
Lista Duplamente Ligada



PLAY / PAUSE



NEXT / PREVIOUS



REPEAT



PLAYLISTS



GRAVAR
PLAYLIST



EQUIPA



Alunos do 2º ano
do curso de
Engenharia
Informática



DOCENTE
Valério Santos



ANO LETIVO
2025/2026

Índice

Introdução.....	3
Objetivos.....	4
Objetivos Específicos	
Ferramentas Utilizadas	
Metodologia.....	5
Diagrama de Atividades.....	6
Diagrama de Caso.....	7
Estrutura de dados.....	8
Estrutura principal da lista duplamente ligada.....	9
Método Adicionar música	
Método Remover Música.....	10
Método Listar.....	11
Método Next	
Método Previous.....	12
Funcionalidades Implementadas	
Interface principal do leitor MP3.....	13
Reprodução de uma música	
Testes Realizados.....	14
Dificuldades encontradas	
Conclusão.....	17
Referencias.....	18

Introdução

Com o avanço das tecnologias digitais e o crescente consumo de conteúdos multimédia, os leitores de música tornaram-se aplicações fundamentais para a organização e reprodução de ficheiros de áudio. Este projeto teve como objetivo desenvolver um leitor MP3 em Java, utilizando uma lista duplamente ligada para a gestão eficiente de playlists.

Durante o desenvolvimento foram aplicados conceitos de Programação Orientada a Objetos, Estruturas de Dados, UML, GitHub e desenvolvimento colaborativo. O sistema permite reproduzir músicas, navegar entre faixas e gerir playlists através de uma interface intuitiva.

```
leitor-mp3-java/  
├── src/  
│   ├── model/    ← DEV BACKEND trabalha aqui  
│   └── ui/        ← DEV GUI trabalha aqui  
├── tests/         ← QA trabalha aqui  
├── docs/  
│   ├── diagramas/ ← DOCUMENTAÇÃO trabalha aqui  
│   ├── relatorio/  ← DOCUMENTAÇÃO trabalha aqui  
│   └── slides/     ← DOCUMENTAÇÃO trabalha aqui  
├── config/        ← ficheiros de configuração  
├── resources/     ← imagens e ícones  
├── README.md      ← este ficheiro  
└── .gitignore     ← diz ao Git o que ignorar
```

Figura 1 – Estrutura geral do projeto no ambiente de desenvolvimento.

Objetivos

Objetivo Geral:

Desenvolver uma aplicação funcional de **Leitor de MP3 em Java**, utilizando uma **lista duplamente ligada** para a gestão de playlists, permitindo ao utilizador reproduzir, organizar e controlar ficheiros de áudio através de funcionalidades como reprodução, pausa, avanço, retrocesso, repetição e gravação de playlists, tanto em modo consola como em interface gráfica.

Além disso, o projeto pretende consolidar os conhecimentos sobre estruturas de dados, programação orientada a objetos e desenvolvimento colaborativo de software, aproximando os estudantes das práticas utilizadas no ambiente profissional.

Objetivos Específicos

- ✓ Implementar um leitor de ficheiros MP3
- ✓ Criar uma estrutura de lista duplamente ligada.
- ✓ Permitir criar e gerir playlists
- ✓ Implementar funções de reprodução, pausa, música seguinte e anterior.
- ✓ Implementar repetição e gravação de playlists
- ✓ Desenvolver uma interface em modo consola e uma interface gráfica
- ✓ Aplicar práticas de trabalho em equipa

Ferramentas Utilizadas

- ✓ Linguagem: Java
- ✓ Controlo de versões: Git e GitHub
- ✓ Biblioteca para reprodução MP3

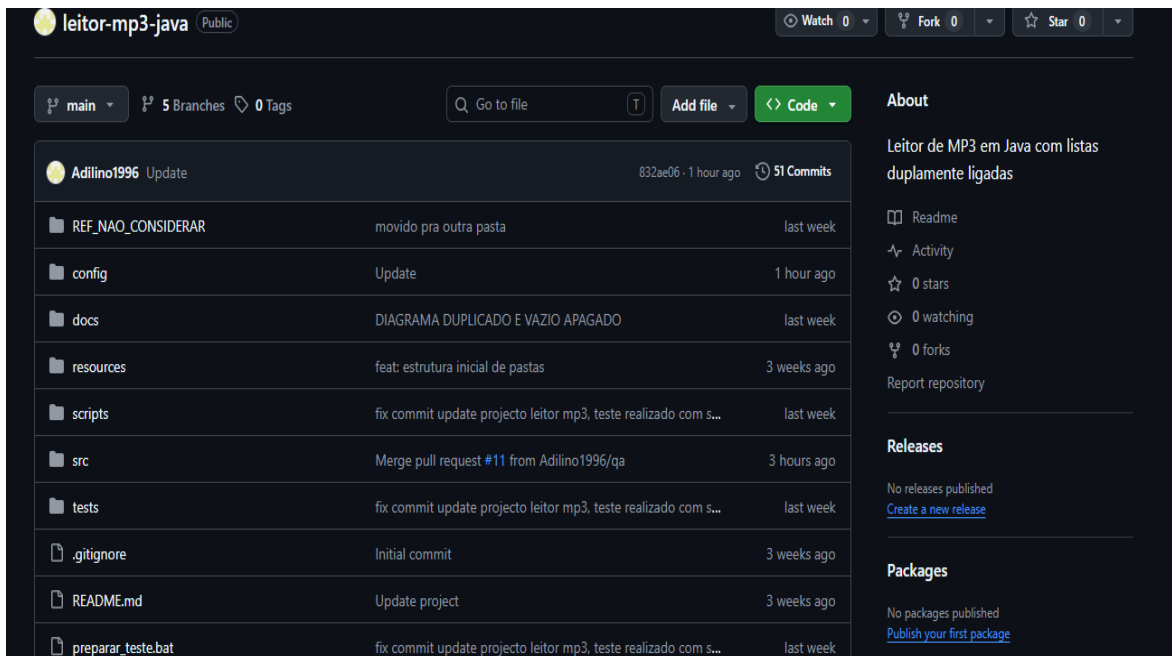


Figura 2 – Repositório GitHub utilizado no desenvolvimento

Metodologia

O desenvolvimento foi dividido em etapas:

1. Formação de equipa e distribuição de tarefas
2. Configuração das ferramentas de trabalho
3. Pesquisa sobre bibliotecas de leitura MP3
4. Criação do diagrama usando draw.io
5. Implementação da lista duplamente ligada
6. Testes e correções de erros
7. Preparação da documentação e apresentação

	DEV GUI (Interface gráfica e modo console)
	Kely Furtado
	Ridney Alves
	Jean Rizac
	Davitson Cardoso
	Jason Varela
	QA / Testes (Testes funcionais e correção de bugs)
	Keila Furtado
	Elizeu Semedo
	Leeliana Barros
	Documentação (Relatório, diagramas e slides)
	Diego da Cruz
	Ivaldina Moeda
	Sara Borges
	Kevin Miranda
	Hélder Tavares
	Elcio Tavares
	Líder (Coordenação geral e gestão do projeto)
	David dos santos
	Igor Gomes

Figura 3-Organização das tarefas da equipa

Diagramas:

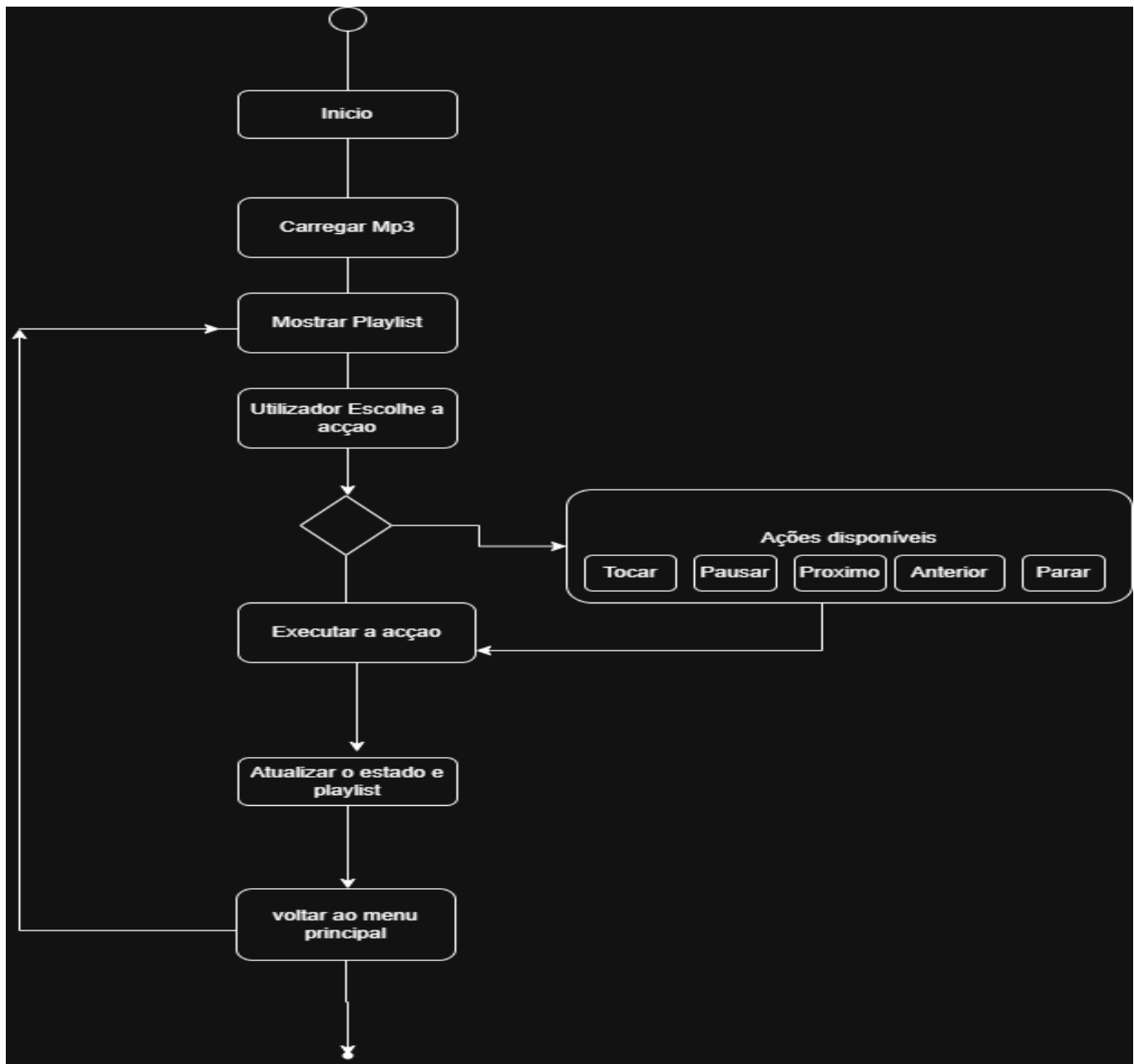


Figura 4- Diagrama de Atividades

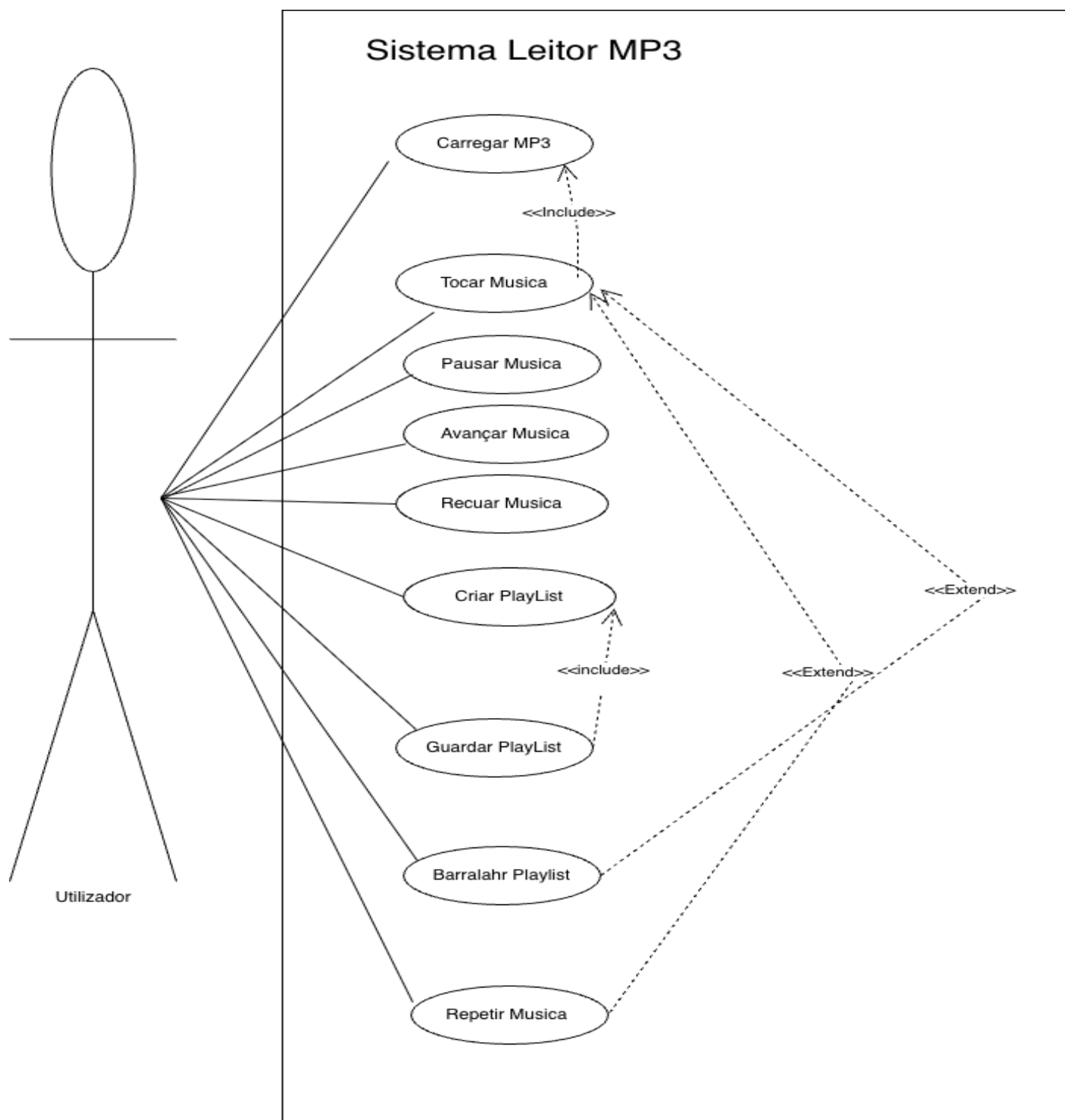


Figura 5: Diagrama de Caso

Estrutura de Dados

Foi utilizada uma lista duplamente ligada.

```
package model;

public class NoMusica {
    private String tituloMusica;
    private String artistaMusica;
    private String caminhoFicheiro;
    private NoMusica anterior;
    private NoMusica proximo;

    public NoMusica(String tituloMusica, String artistaMusica, String caminhoFicheiro) {
        this.tituloMusica = tituloMusica;
        this.artistaMusica = artistaMusica;
        this.caminhoFicheiro = caminhoFicheiro;
        this.anterior = null;
        this.proximo = null;
    }

    public String getTituloMusica() {
        return tituloMusica;
    }

    public void setTituloMusica(String tituloMusica) {
        this.tituloMusica = tituloMusica;
    }

    public String getArtistaMusica() {
        return artistaMusica;
    }

    public void setArtistaMusica(String artistaMusica) {
        this.artistaMusica = artistaMusica;
    }

    public String getCaminhoFicheiro() {
        return caminhoFicheiro;
    }

    public void setCaminhoFicheiro(String caminhoFicheiro) {
        this.caminhoFicheiro = caminhoFicheiro;
    }

    public NoMusica getAnterior() {
        return anterior;
    }

    public void setAnterior(NoMusica anterior) {
        this.anterior = anterior;
    }

    public NoMusica getProximo() {
        return proximo;
    }

    public void setProximo(NoMusica proximo) {
        this.proximo = proximo;
    }

    @Override
    public String toString() {
        return "Titulo: " + tituloMusica +
            ", Artista: " + artistaMusica +
            ", Ficheiro: " + caminhoFicheiro;
    }
}
```

Figura 6-Classe NoMusica


```

package model;

public class PlaylistDupla {
    private NoMusica inicio;
    private NoMusica fim;
    private NoMusica atualMusica;
    private int tamanho;

    public PlaylistDupla() {
        this.inicio = null;
        this.fim = null;
        this.atualMusica = null;
        this.tamanho = 0;
    }

    public NoMusica getAtualMusica() {
        return atualMusica;
    }
}

```

Figura 7-Estrutura principal da lista duplamente ligada

```

public void adicionarMusica(String tituloMusica,String artistaMusica,String caminhoFicheiro){
    //criar um NO
    NoMusica novaMusica = new NoMusica(tituloMusica, artistaMusica, caminhoFicheiro);
    //se a lista estiver vazia quer dizer que o NO a inserir sera o primeiro NO e o ultimo NO e nao tera um anterior e proximo
    if (isVazia()) {
        this.inicio = novaMusica;
        this.fim = novaMusica;
        this.atualMusica = novaMusica;
    }else{
        //vamos ligar o ponteiro do ultimo No ao novo no
        fim.setProximo(novaMusica);
        //ligar o ponteiro do novo No que agora é o ultimo ao No que era o ultimo no
        novaMusica.setAnterior(fim);
        //atualizar o valor do fim
        this.fim = novaMusica;
    }
    this.tamanho++;
}
}

```

Figura 8-Método Adicionar música

```

public void removerMusica(String titulo){
    if (isVazia()) {
        System.out.println("Lista esta vazia");
        return;
    }

    NoMusica cursor = inicio;
    NoMusica encontrado = null;
    while (cursor != null) {
        if (cursor.getTituloMusica().equalsIgnoreCase(titulo)) {
            encontrado = cursor;
            break;
        }
        cursor = cursor.getProximo();
    }

    if (encontrado == null) {
        System.out.println("Musica nao encontrada");
        return;
    }

    if (inicio == fim) {
        inicio = null;
        fim = null;
        atualMusica = null;
        this.tamanho--;
        return;
    }

    if (encontrado == inicio) { //caso for primeiro no
        NoMusica segundoNo = encontrado.getProximo();
        this.inicio = segundoNo;
        segundoNo.setAnterior(null);
        if (encontrado == atualMusica) { //caso musica a eliminar for atual musica
            atualMusica = segundoNo;
        }
        this.tamanho--;
        return;
    }

    if (encontrado == fim) { //caso for ultimo no
        NoMusica penultimo = encontrado.getAnterior();
        this.fim = penultimo;
        penultimo.setProximo(null);
        if (encontrado == atualMusica) {
            atualMusica = penultimo;
        }
        this.tamanho--;
        return;
    }

    NoMusica anterior = encontrado.getAnterior();
    NoMusica proximo = encontrado.getProximo();
    anterior.setProximo(proximo);
    proximo.setAnterior(anterior);
    if (encontrado == atualMusica) {
        atualMusica = proximo;
    }
    this.tamanho--;
}

```

Figura 9-Método Remover Música

```

public void listar(){
    if (isVazia()) {
        System.out.println("Lista esta vazia");
        return;
    }

    NoMusica cursor = inicio;
    int contador = 1;
    while (cursor != null) {
        if (cursor == atualMusica){
            System.out.println(contador + " - " + cursor + " <-- TOCANDO AGORA");
        }else{
            System.out.println(contador + " - " + cursor);
        }
        cursor = cursor.getProximo();
        contador++;
    }
}

```

Figura 10-Método Listar

```

public void next(){
    if (atualMusica == null) {
        System.out.println("Nao existe musica para avançar");
        return;
    }

    if (atualMusica.getProximo() != null) {
        atualMusica = atualMusica.getProximo();
    }else{
        atualMusica = inicio;
    }
    System.out.println("Musica a tocar : " + atualMusica);
}

```

Figura 11-Método Next

```
public void previous(){
    if (atualMusica == null) {
        System.out.println("Nao existe musica atual");
        return;
    }

    if (atualMusica.getAnterior() != null) {
        atualMusica = atualMusica.getAnterior();
    }
    System.out.println("Musica a tocar : " + atualMusica);
}
```

Figura 12-Método Previous

Com este tipo de estrutura permite navegar facilmente entre as músicas anteriores e seguintes

Funcionalidades Implementadas

- ✓ Carregar música
- ✓ Criar playlist
- ✓ Reproduzir música
- ✓ Pausar reprodução
- ✓ Avançar para próxima música
- ✓ Volta para a música anterior
- ✓ Repetir música
- ✓ Guardar playlist

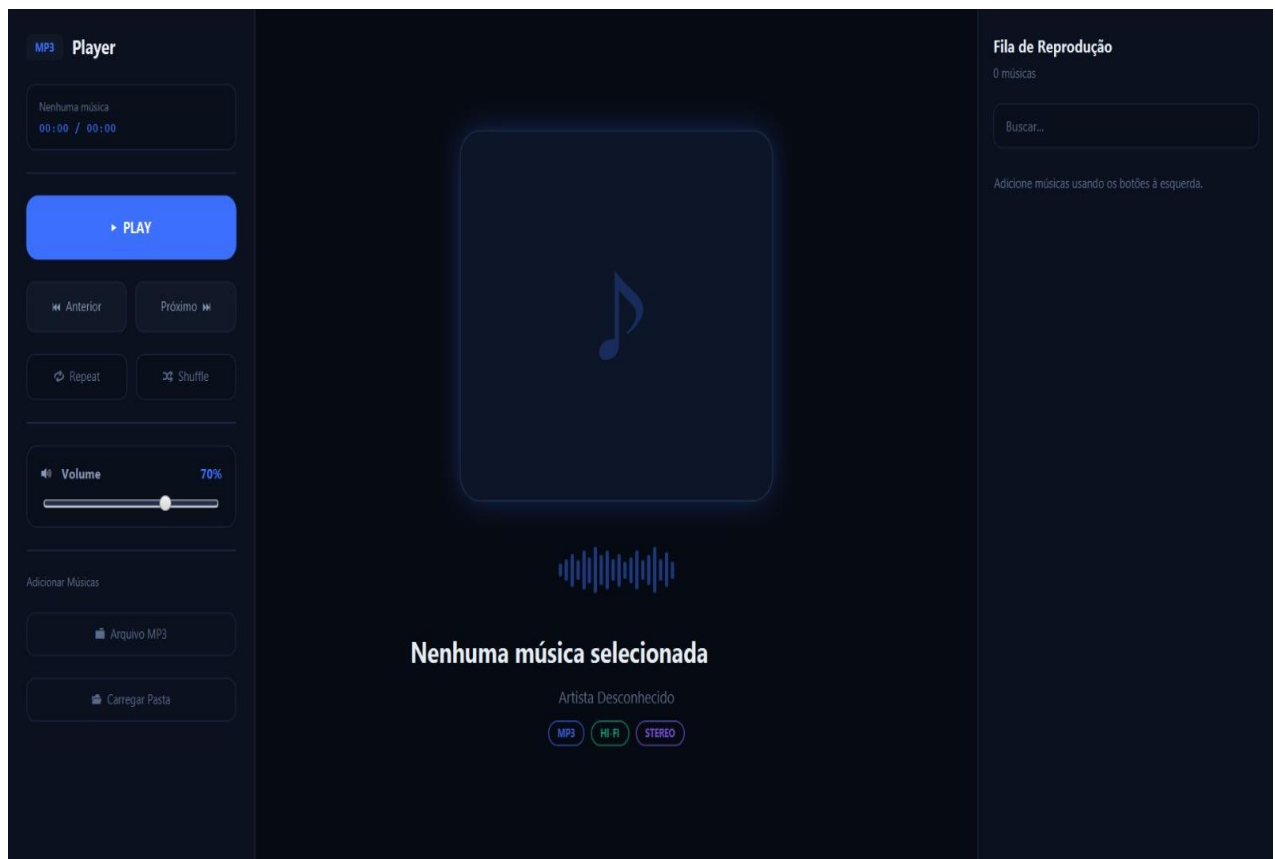
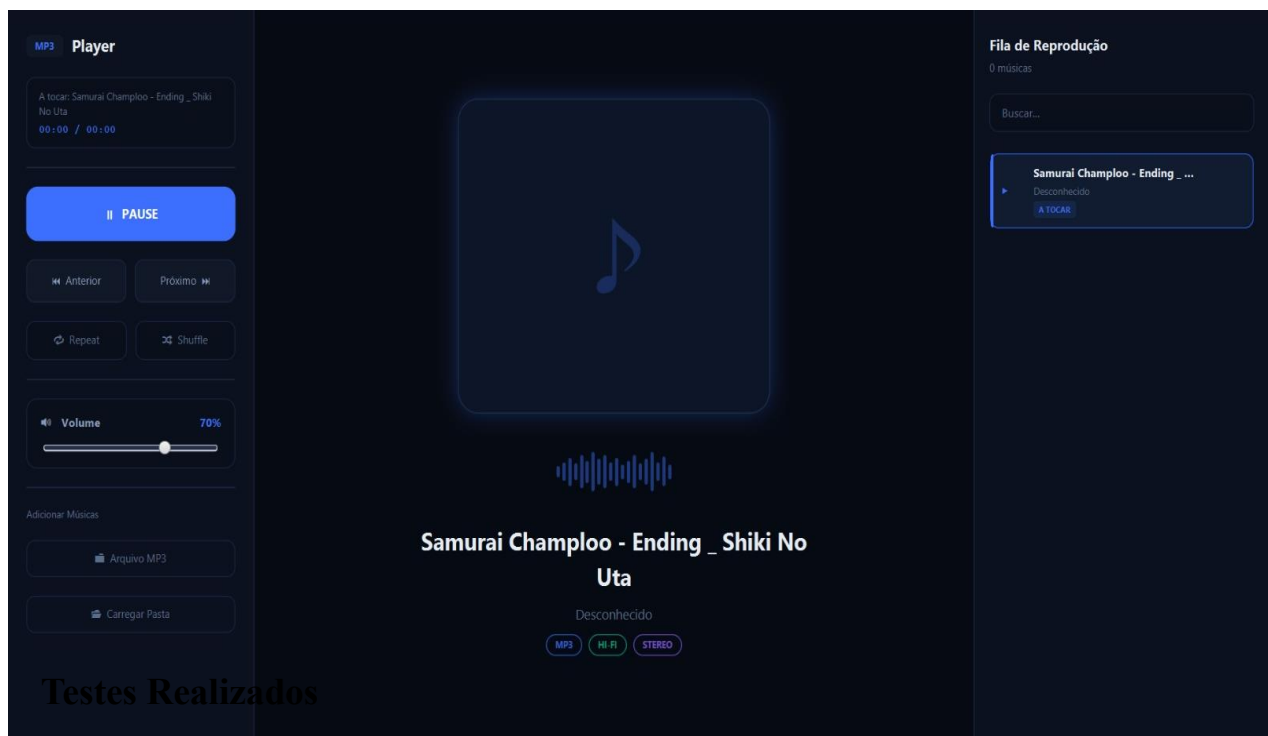


Figura 13-Interface principal do leitor MP3.



Testes Realizados

Figura 14- Reprodução de uma música

Foram realizados testes para verificar:

- ✓ Carregamento correto das músicas
- ✓ Funcionamento da playlist
- ✓ Navegação entre músicas
- ✓ Funcionamento do modo Repeat
- ✓ Funcionamento do Play/Pause
- ✓ Gravação e leitura da playlist

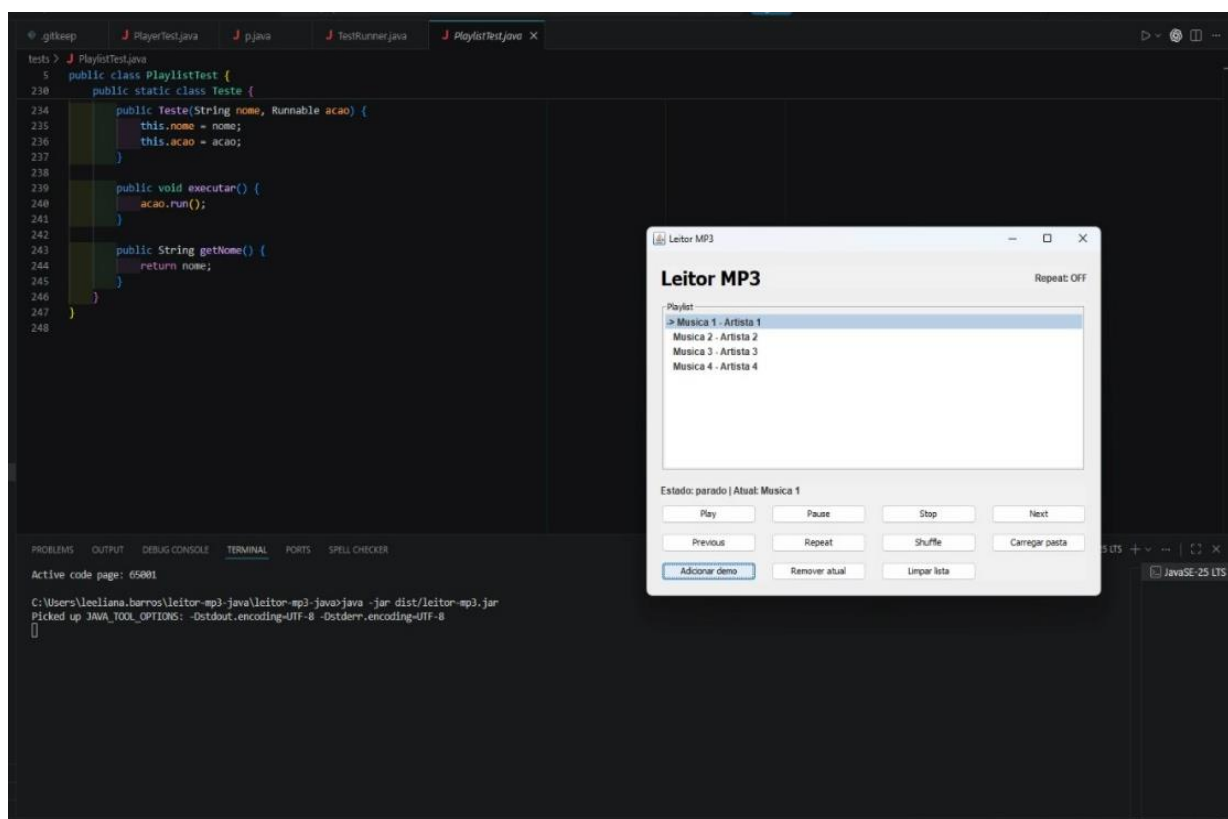


Figura 15 – Teste de inserção de músicas

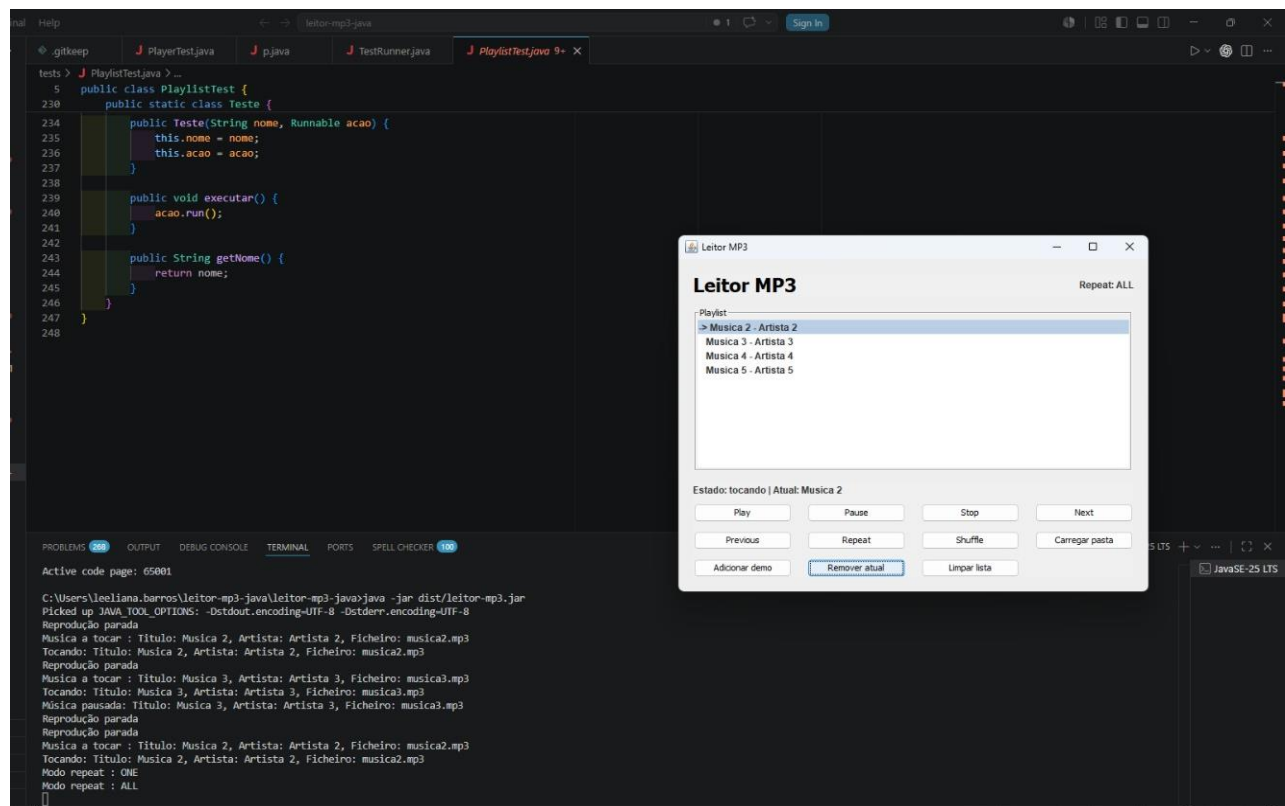


Figura 16 – Teste de remoção de músicas

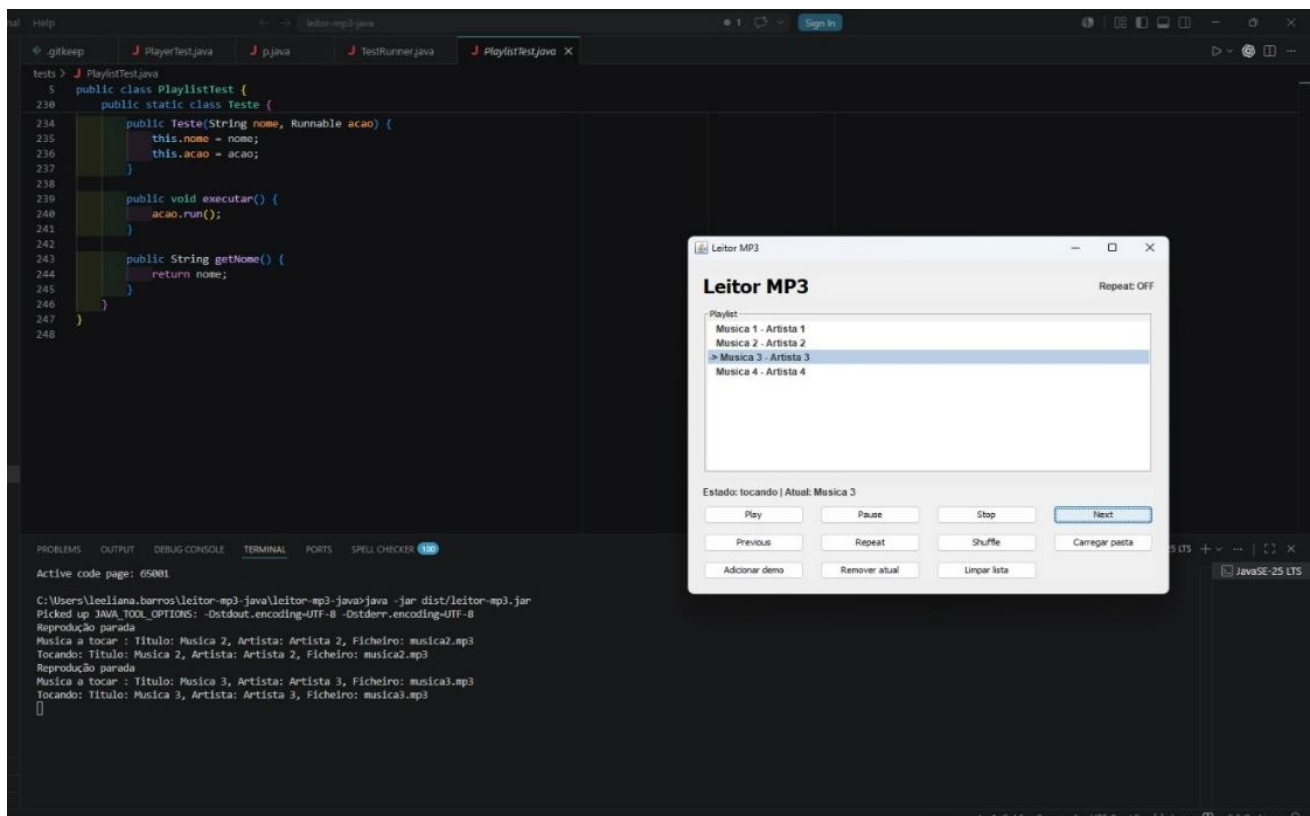
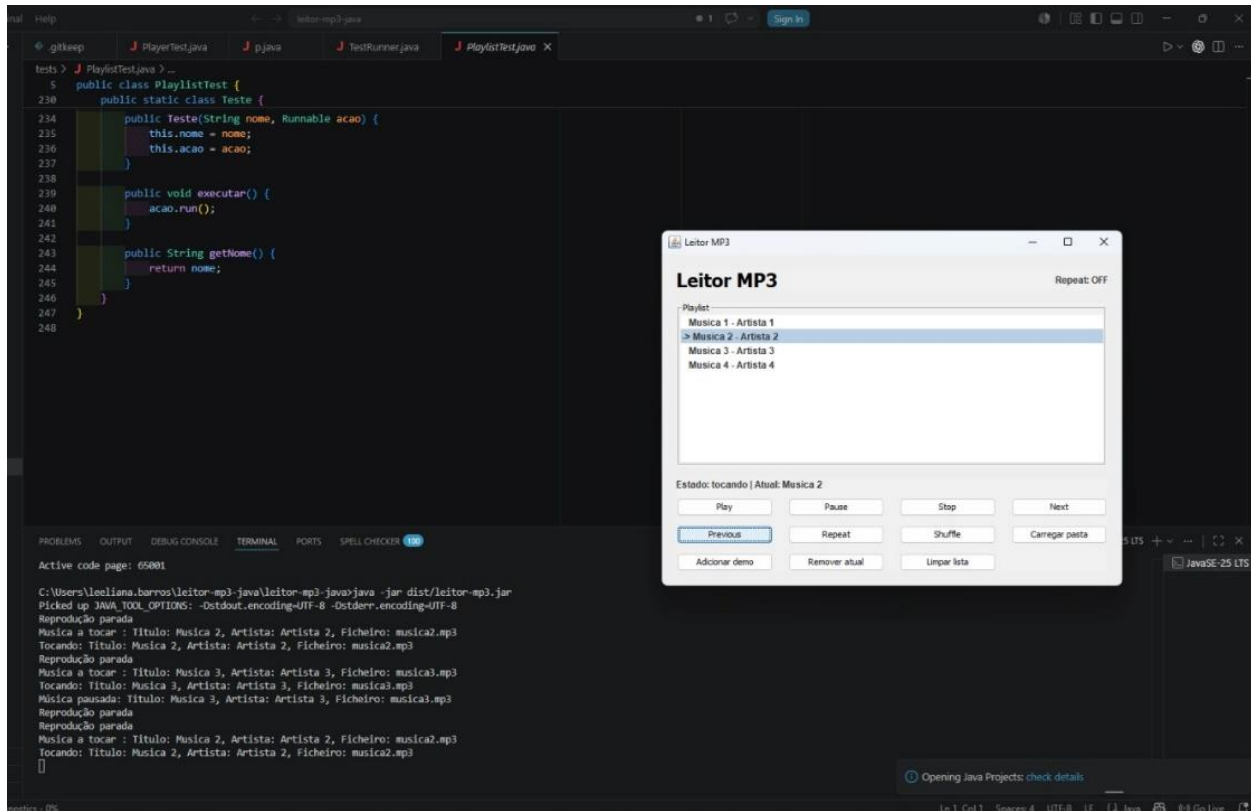


Figura 17 – Teste da funcionalidade Next



Os testes mostram que as funcionalidades principais operam corretamente

As principais dificuldades encontradas estiveram relacionadas com a implementação da lista duplamente ligada, integração da reprodução MP3 e sincronização entre a interface gráfica e a playlist. Estas dificuldades foram ultrapassadas através de pesquisa, testes e trabalho colaborativo.

Conclusão

A realização deste trabalho permitiu colocar em prática diversos conceitos estudados ao longo da disciplina, demonstrando a importância das estruturas de dados e da programação orientada a objetos no desenvolvimento de aplicações reais. A implementação da lista duplamente ligada mostrou-se uma solução eficiente para a gestão das playlists, facilitando a navegação entre músicas e a organização dos ficheiros.

O desenvolvimento do leitor de MP3 proporcionou também experiência na manipulação de bibliotecas Java, na criação de interfaces, na utilização de ferramentas de controlo de versões e na colaboração entre os membros da equipa. Durante o processo surgiram desafios técnicos, que foram superados através de pesquisa, testes e correção de erros, contribuindo para o aperfeiçoamento das competências dos participantes.

Conclui-se que os objetivos propostos foram alcançados, resultando numa aplicação funcional e de valor académico. O projeto reforçou a importância do planeamento, da organização e do trabalho em equipa, além de servir como base para futuras melhorias.

Bibliografia

<https://github.com/Adilino1996/leitor-mp3-java/tree/main/docs/diagramas>

<https://elearning.us.edu.cv/mod/forum/discuss.php?d=2469>

<https://chatgpt.com/c/6a2beb53-5660-83ea-ad5a-b98853abb08d>

<https://github.com/Adilino1996/leitor-mp3-java/blob/main/src/model/PlaylistDupla.java>

<https://github.com/Adilino1996/leitor-mp3-java/blob/main/src/model/NoMusica.java>